Index

S.NO	Topic	marks	Sign
1.	Caesar Cipher	10	
2.	Monoalphabetic Substitution Cipher	10	
3.	Playfair Cipher	10	
4.	Polyalphabetic Substitution Cipher.	10	
5.	Affine Caesar Cipher	10	
6.	Breaking an Affine Cipher	10	
7.	Decrypt a simple Substitution Cipher	10	
8.	Keyword Monoalphabetic Cipher	10	
9.	Playfair cipher Decryption.	10	
10.	Playfair Cipher Encryption.	10	
11.	No. of possible Playfair keys	10	
12.	Hill cipher Encryption & Decryption	10	
13.	Known Plaintext Attack on hill cipher	10	
14.	One-Time Pad Encryption & Decryption	10	
15.	Letter Frequency Attack on Additive Cipher.	10	
16.	Mono attack	10	
17.	DES Decryption		

S.NO	Title	Marks	Sign
18.	DES subkeys	10	
19.	CBC 3DES	10	
20.	ECBCBC error	10	
21.	Padding modes	10	
22.	CBC encryption	10	
23.	Counter mode	10	
24.	RSA key	10	
25.	RSA Factor	10	
26.	RSA key update	10	
27.	RSA Attack	10	
28.	Diffie Hellman	10	
29.	SHA3	10	
30	CBC MAC	10	
31.	CMAC subkey Generation	10	
32.	DSA signature	10	
33.	DES Implementation	10	
34.	Padding	10	
35.	One Time Pad Vignere	10	
36.	Affine Caesar Cipher	10	
37.	Monoalphabetic frequency attack	10	
38.	Hill-cipher known	10	
39.	Additive Cipher frequency attack	10	
40.	Monoalphabetic frequency attack	10	

Exp 16:- Monoattack

Aim :- To write a C program to perform a letter frequency attack on a Monoalphabetic substitution cipher and Generate possible pailnterts.

Algorithm :-

1. Read the ciphertext from the user
2. Count the frequency of each alphabet letter.
3. Arrange the letters in decreasing order.
4. Display the top 10 frequent letters.
5. Use the frequency order to estimate plaintext
6. STOP

Output :- Enter Cipher text :

WKLV LVD WHVW PHVVDJH

1) V=4
2) W=3
3) H=3
4) L=2
5) D=2
6) K=1
7) P=1
8) J=1
9) A=0
10) B=0

Result :- Thus, the program was executed successfully.

Exp 17 : DES Decryption.

Aim :- To write a 'C' program to generate DES decryption round keys by reversing the order of the encryption keys.

Algorithm :-

1. Read the 16 encryption round keys
2. store the keys in an array
3. Reverse the order of keys
4. Display the decryption round keys.
5. STOP.

Output :- Enter the 16 Encryption round keys:

1 2 3 4 5 6 7 8

DES Decryption

$K_1 = 8$ $K_5 = 4$

$K_2 = 7$ $K_6 = 3$

$K_3 = 6$ $K_7 = 2$

$K_4 = 5$ $K_8 = 1$

Result :- Thus, the program executed successfully

Exp 18 :- DES Subkey

Aim:- To write a C program to demonstrate DES subkey generation by dividing the key into two 28-bit halves.

Algorithm:-

1. Read the 56-bit key
2. Divide it into left and right 28-bit halves
3. Generate 16 subkeys using circular
4. Display the generated subkeys.
5. STOP

Enter key:

1010101010101010

left half : 1010101

right half : 1010101

Generated DES subkeys:

K_1 = 10101010101010

K_2 = 10101010101010

⋮

K_{16} = 10101010.

Result:- Thus the program was executed Successfully.

Exp 19 : CBC3 DES.

Aim :- To write a C program to demonstrate cipher block chaining (CBC) mode using the triple DES (3DES) algorithm.

Algorithm :-

1. Read the plaintext
2. Read the key and Initialization vector (IV)
3. Encrypt the plaintext using 3DES in CBC mode
4. Display the cipher text
5. STOP

Output :-

Enter PT : HELLO WORLD

cipher text : (3DES CBC mode) :

8F2A6CAD4B1E7F35.

Result :- Thus, the program executed successfully

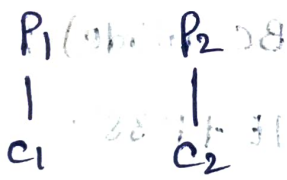
Exp no : ECBCB Error

Aim :- To write a C program to demonstrate error propogation in ECB and CBC modes of operation.

Algorithm :-

1. Read the transmission error condition.
2. Check the effort ciphertext block
3. Display the effect in ECB mode
4. Display the effect in CBC mode
5. Print the conclusion.
6. STOP

Output :-



After error C_1

ECB: P_1 affected

P_2, P_3, P_4 - unaffected

CBC:

P_1 affected

P_2 affected

P_3, P_4 unaffected

Result :- Thus, the program executed successfully.

Exp 21 : Padding Mode

Aim:- To write a C program to demonstrate Padding in ECB, CBC and CFB modes of operation

Algorithm:-

1. Read the plaintext from the user
2. Check whether the PT length is a multiple of the block size.
3. Add padding (1 followed by 0s) if required
4. Display the padded plaintext
5. STOP

Output:-

```
enter plaintext : HELLO
original length : 5
Padded plaintext : HELLO 000
```

Result:- Thus the program was executed Successfully

Exp 22 CBC Encryption

Aim

To write a program to encrypt and decrypt data using cipher block chaining (CBC) mode.

Algorithm

1. Read the plaintext, key and initialization vector
2. XOR the plaintext with the IV.
3. Encrypt the ciphertext using the same key.
4. Decrypt the ciphertext using the same key.
5. Display the plaintext, ciphertext, and decrypted text.
6. Stop

output:

plaintext : 23
IV : AA
Key : FD

plaintext : 23
IV : AA
Key = FD

ciphertext : 7H

Decrypted : 23

Result:

Thus the program was executed successfully.

Exp 23 :- Counter Mode

Aim :- To write a C program to perform encryption and decryption using counter (CTR) mode.

Algorithm :-

1. Read the plaintext, key and counter.
2. Encrypt the counter using the key.
3. XOR the encrypted counter with the plaintext to get the cipher text.
4. Repeat for each block & Increment the counter.
5. Decrypt using the same process
6. STOP

Output :-

Plaintext : 23

Counter : 00

key : FD

Ciphertext : DE

Decryption : 23

Result :- Thus, the program was executed successfully.

Exp 24 :- RSA key

Aim :- To write a C program to determine the RSA private key from the given public key

Algorithm :-

1. Read the values of e and n
2. Find the prime factors P and q
3. Compute $\phi(n) = (P-1)(q-1)$
4. Find d such that $(d \times e) \bmod \phi(n) = 1$
5. Display the private key.
6. STOP

Output :-

Public key = (31, 3599)

Private key = (3431, 3599)

Result :- Thus, the program was executed Successfully.

Exp 25 RSA factor

Aim

To write a C program to demonstrate factorization in RSA.

Algorithm

1. Read n and the common factor
2. Find the other factor by division
3. Display both prime factors
4. Explain the impact on RSA
5. Stop the program

output

Known factor: 59

other factor: 61

RSA is broken because n is factored.

Result:

Thus the program has been executed successfully.

Exp 26 : RSA key update

Aim : To write a C program to study RSA key regeneration after a private key leak

Algorithm :-

1. Read the public and private key
2. Assume the private key is leaked
3. Generated new keys using the same modules
4. Display the new keys
5. Explain whether it is secure
6. Explain whether it is valid.

Output :-

RSA Key Regeneration.

Old modules : n

Old pulbic key : e

old private key = d.

Generating new public and private keys using same modules...

Results, Not safe.

Result :- Thus, the program was executed successfully.

Exp 27 :- RSA Attack.

Aim :- To write a C program to demonstrate the insecurity of encryption each alphabet individuality using RSA

Algorithm :-

1. Read a plaintext charecter
2. Convert it to a number (A=0, B=1 ...)
3. Encrypt it using RSA
4. Display the ciphertext
5. Explain the Attack
6. STOP

Output :-

Enter charecter : H

Ciphertext : 182

Attack : Dictionary (Brute Force) attack.

Result :- Thus the program executed successfully.

Exp 28 :- Diffie Hellman

Aim :- To write a C program to demonstrate the diffie Hellman key exchange.

Algorithm :-

1. Read public values a and q
2. Read Alice's and Bob's secret keys
3. Compute public keys.
4. Compute the shared secret
5. Display the shared key:
6. STOP.

Output :-

Allice key = 2

Bob key = 2

Result :- Thus, the program was executed Successfully.

Exp 29:- SHA3

Aim:- To write a C program to demonstrate SHA-3 lane Initialization

Algorithm:-

1. Initialization the state matter
2. Set capacity lanes to zero
3. Process the message block
4. Count Non-zero lanes
5. Display the result
6. STOP

Output:-

Total lanes : 25

Rate lanes : 10

Capacity lanes : 9

After one permutation all capacity lanes become Non-zero.

Result:- Thus, the program was executed successfully

Exp 30 :- CBC MAC

Aim:- To write a C program to demonstrate CBC-MAC generation.

Algorithm:-

1. Read the message block
2. generate the CBC-MAC
3. Append XOR to the message
4. Generate the new MAC
5. Display the result
6. STOP.

Output :- Message X = 25

MAC T = 179

XOR T = 170

CBC-MAC of (X XOR T) is again T

Result :- Thus, the program was executed successfully.